

Problem Solving Through Programming

CS1001-1

Unit-2

Chapter 5: Decision making and Branching

Decision making with if statement, Simple if Statement, the if...else statement, Nesting of if...else statements, The else...if ladder, The switch statement, The ternary operator, The goto statement, break and continue statements.

Chapter 6: Decision making and Looping

The while statement, the do...while statement, the for statement, Jumps in Loops.

Chapter 7: Arrays

Arrays (1-D, 2-D) Initialization and Declaration. Examples for one dimensional and two dimensional arrays.

Chapter 8: User-defined Functions

Need for the User-defined Functions, Element of User-defined Functions, Definition of Functions, Argument Passing –call by value, call by reference. Return Values and their Types, Function Calls, Function Declaration, Category of Functions.

Chapter 5: Decision making and Branching

C program is a set of statements which are normally executed sequentially in the order in which they appear. This is called ***Sequential Execution***.

This happens when no options or repetitions of certain calculations are necessary.

But, in actual practice, the programmer may have to:

- change the order of execution of statements based on certain conditions
- repeat a group of statements until certain specified conditions are met

This involves a kind of ***decision making*** to see whether a particular condition has occurred or not and then direct the computer to execute certain statements accordingly.

To handle such situations, C provides ***Control Statements*** or ***Decision Making Statements***.

The control statements determine the flow of control in a program.

Types of Control Statements are:

(1) Conditional Branching Control Statements

- (a) **if** statement
- (b) **if-else** statement
- (c) **nested if-else** statement
- (d) **switch** statement

(2) Unconditional Branching Control Statement

- (a) **goto** statement

CONDITIONAL BRANCHING

(A) THE SIMPLE IF STATEMENT

It is a powerful decision making statement used to control the flow of execution of statements. It is called one way branching statement.

It involves a test expression (a relational or conditional expression).

Syntax and flowchart of if statement:

Single statement

```
if (testExpression)
{
    statement;    // body of if
}
statement -x;
```

Compound Statement

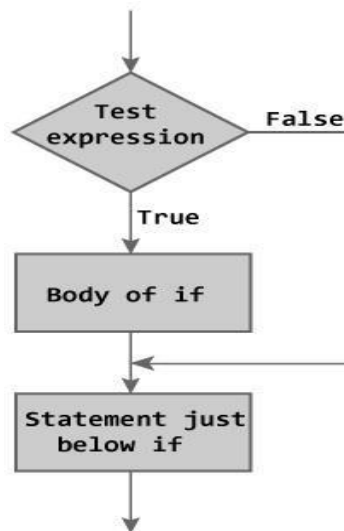
```
if (testExpression)
{
    statement-1;
    statement -2;
    .....
    statement -n;
}
statement -x;
```

The statement associated with an **if** statement may be either a single statement or compound statement.

if statement evaluates the test expression inside the parenthesis.

If the test expression is evaluated to true (nonzero), statements inside the body of if is executed.

If the test expression is evaluated to false (0), a statement inside the body of if is skipped from execution.



Example (1): While purchasing certain items, a discount of 10% is offered if the purchase quantity is more than 1000 items. If quantity and rate are input through the key board, write a program to calculate the total expense.

```
/**USE OF IF STATEMENT (SINGLE STATEMENT)**/  
#include <stdio.h>  
  
int main()  
{  
    int qty;  
    float rate,tot,dis=0;  
    printf("\n\nEnter the quantity and rate:");  
    scanf("%d%f",&qty,&rate);  
    if(qty > 1000) /* 'IF' WITH SINGLE STATEMENT*/  
        dis = qty*rate*10/100;  
    tot = qty*rate - dis;  
    printf("\n\nTotal cost= Rs. %f",tot);  
    printf("\nSavings= Rs. %f",dis);  
    return 0;  
}
```

Output of program:

(1) Enter the quantity and rate:200 15.50

Total cost = Rs. 3100.000000

Savings = Rs. 0.000000

(2) Enter the quantity and rate:1200 15.50

Total cost = Rs. 16740.000000

Savings = Rs. 1860.000000

Example (2): The current year and the year in which the employee joined the organization are entered through the key board. If the number of years for which the employee has served the organization is greater than 3 years, a bonus of Rs. 2500 is given to the employee. If the years of service are less than 3, the program should do nothing.

```
/**USE OF IF STATEMENT (COMPOUND STATEMENT)***/
#include <stdio.h>
#include <stdlib.h>
int main( )
{
    int bonus=0, cryr, yrjn, yrsr;
    printf("\nEnter current year and year of joining:");
    scanf("%d%d",&cryr,&yrjn);
    yrsr = cryr - yrjn;
    if(yrsr > 3) /* 'IF' WITH COMPOUND STATEMENT*/
    {
        bonus = 2500;
        printf("\nBonus = Rs. %d",bonus);
        exit(0);
    }
    printf("\nBonus= Rs. %d",bonus);
    return 0;
}
```

Output of program:

(1) Enter current year and year of joining: 2006 1998

Bonus = Rs. 2500

(2) Enter current year and year of joining: 2006 2004

Bonus = Rs. 0

Example (3): Write a C program to print the number entered by user only if the number entered is negative.

```
#include<stdio.h>
int main()
{
    int num;
    printf("Enter a number to check.\n");
    scanf("%d",&num);
    if(num<0)
        printf("Number=%d\n",num);
    printf("The if statement in C programming is easy.");
    return 0;
}
```

Output 1

Enter a number to check.
-2
Number=-2
The if statement in C programming is easy.

Output 2

Enter a number to check.
5
The if statement in C programming is easy.

(B) THE IF-ELSE STATEMENT

The **if** statement provides one way branching, i.e., it executes the statement/s associated with **if**, only when the test expression is true, but does nothing if it is false.

The **if-else** statement is an extension of the **if** statement. The **if-else** statement is a *two way*

branching, i.e., it executes one set of statement/s if the test expression is true or it executes another set of statement/s if the test expression is false.

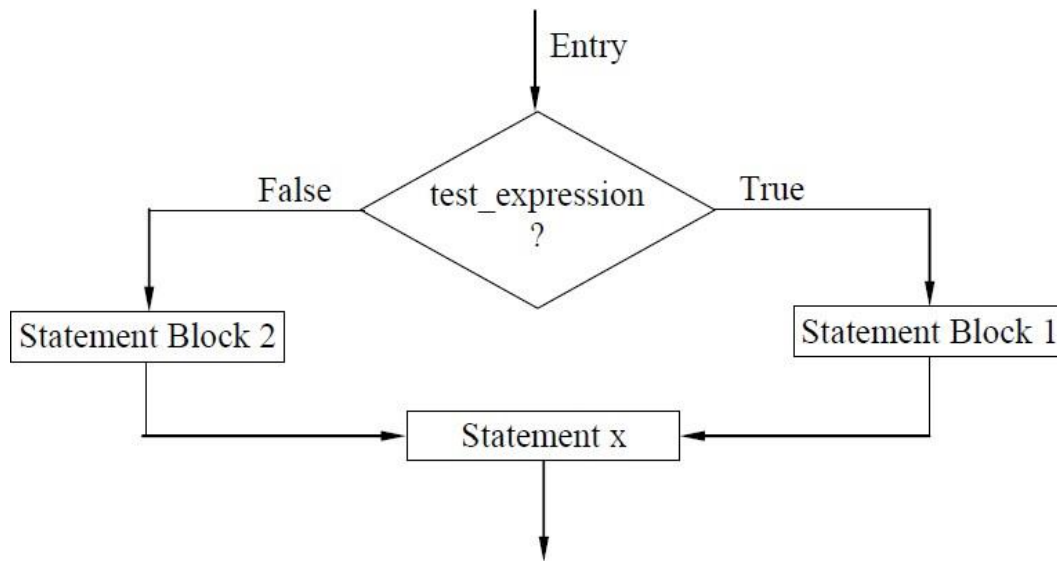
Syntax of the if-else statement:

if-else with single statement

```
if(test_expression)
    statement_true;
else
    statement_false;
statementx;
statementy;
```

if-else with compound statement

```
if(test_expression)
{
    statement 1;
    statement 2;
    statement 3;
}
else
{
    statement 4;
    statement 5;
    statement 6;
}
statement x;
statement y;
```

Example: If a person's basic salary is less than Rs. 10000, then HRA = 10% of basic salary and DA = 25% of basic salary. If his salary is equal to or above Rs. 10000, then HRA = Rs. 4000 and DA = 50% of basic salary. If the employee's salary is input through the key board, write a program to find his gross salary.

```

/****USE OF IF-ELSE STATEMENT (COMPOUND STATEMENT)****/
#include <stdio.h>

int main()
{
    float basic,gross,da,hra;
    printf("\n\nEnter    basic salary:");
    scanf("%f",&basic);
    if(basic >= 10000) /* 'IF-ELSE'    WITH COMPOUND STATEMENT*/
    {
        hra = 4000;
        da = basic*50/100;
    }
}
  
```

```

}
else
{
    hra = basic*10/100;
    da = basic*25/100;
}
gross = basic + hra + da;
printf("\nGross Salary is Rs. %f",gross);
return 0;
}

```

Output of program:

(1) Enter basic salary:8000

Gross Salary is Rs.10800.000000

(2) Enter basic salary:12000

Gross Salary is Rs.22000.000000

(C) NESTED IF STATEMENTS

When a series of decisions are involved, we may have to use more than one **if-else** statement.

Enclosing one **if** within another is called **nested if** statement.

Syntax:

```

if (condition1)
{
    if (condition2)
    {
        statement1;
    }
}

```

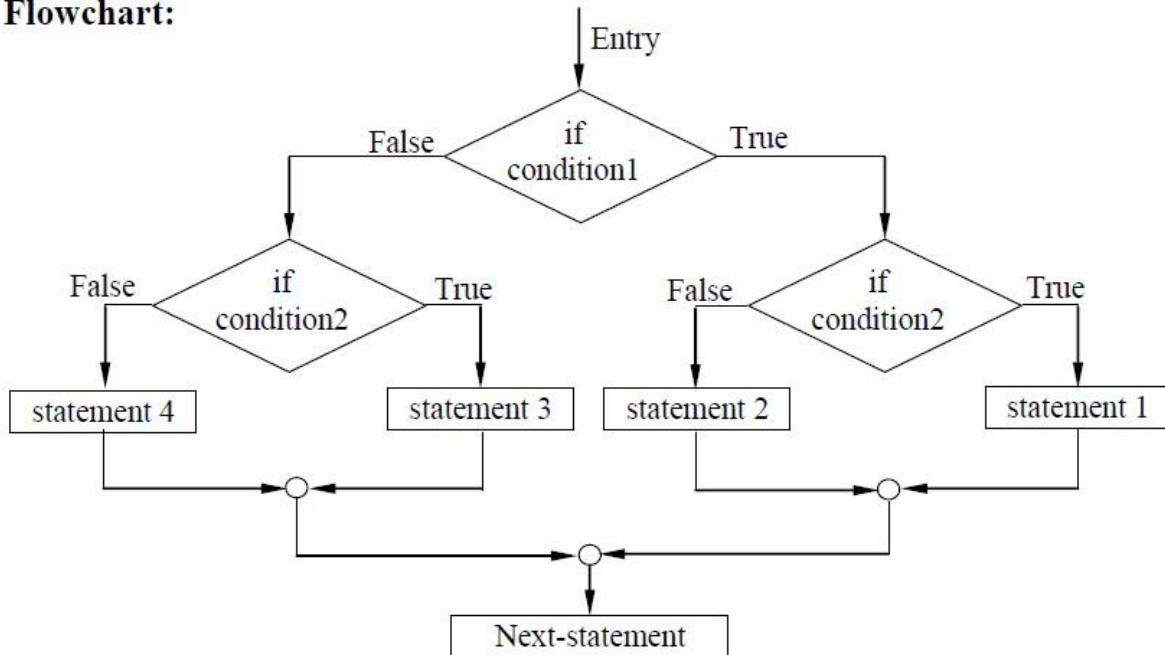
```

    }
    else
    {
        statement2;
    }
}
else
{
    if (condition3)
    {
        statement3;
    }
    else
    {
        statement4;
    }
}

```

1. If **condition1** is TRUE and **condition2** is also TRUE, then **statement 1** is executed.
2. If **condition1** is TRUE and **condition2** is FALSE, then **statement 2** is executed.
3. If **condition1** is FALSE and **condition3** is TRUE, then **statement 3** is executed.
4. If **condition1** is FALSE and **condition3** is also FALSE, then **statement 4** is executed.

Flowchart:



Example 1: Write a C program to read values for 3 sides of a triangle, check if the triangle is equilateral, isosceles or scalene and report it.

```
/**USE OF NESTED IF***/  
#include<stdio.h>  
int main()  
{  
float s1,s2,s3;  
printf("\n\nEnter the 3 sides of triangle:");  
scanf("%f%f%f",&s1,&s2,&s3);  
if((s1+s2)>s3&&(s2+s3)>s1&&(s3+s1)>s2)  
{  
printf("\n\nGiven 3 sides form a triangle...");  

```

```

    if(s1==s2 && s2==s3)
printf("\nIt is an Equilateral triangle");
else
if(s1==s2||s2==s3||s3==s1)
printf("\nIt is an isosceles triangle");
else
printf("\nIt is a scalene triangle");
}
else
printf("\n\nSorry...entered 3 sides do not form a triangle...");
return 0;
}

```

Example 2: Write a C program to relate two integers entered by user using = or > or < sign.

```

#include <stdio.h>
int main()
{
    int numb1, numb2;
    printf("Enter two integers to check".\n);
    scanf("%d %d",&numb1,&numb2);
    if(numb1==numb2)
        printf("Result: %d=%d",numb1,numb2);
    else
        if(numb1>numb2)
            printf("Result: %d>%d",numb1,numb2);
        else

```

```
        printf("Result: %d>%d",numb2,numb1);
return 0;
}
```

Output 1

Enter two integers to check.

5

3

Result: 5>3

Output 2

Enter two integers to check.

-4

-4

Result: -4=-4

(D) THE ELSE-IF LADDER

When multipath decisions are involved, we can have a chain of **ifs** in which the statements associated with each **else** is an **if**.

Syntax:

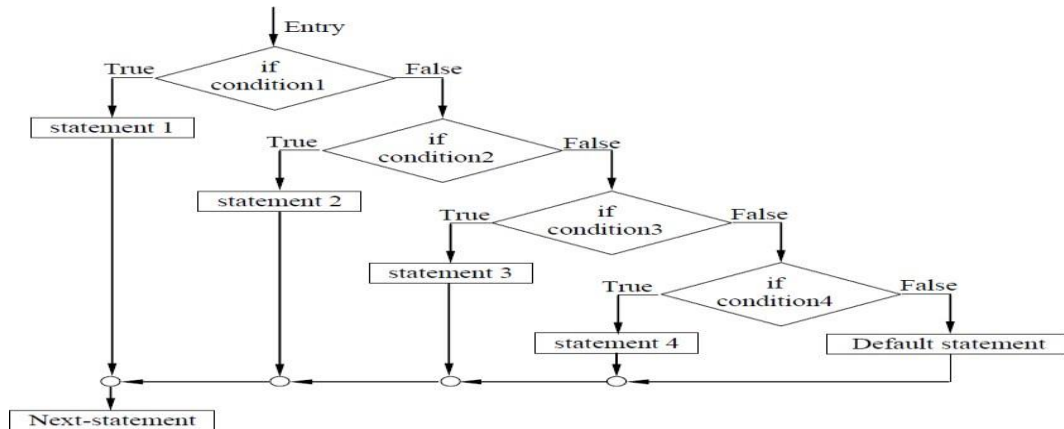
```
if(condition1)
    statement 1;
else if(condition2)
    statement 2;
else if(condition3)
    statement 3;
else if(condition4)
    statement 4;
-----
```

```

-----
if(condition n)
    statement n;
else
    default-statement;

```

Flowchart:



Example1: Consider the problem of grading students in academic institutions.

Average marks	Grade
80-100	A
60-79	B
50-59	C
40-49	D
0-39	F

```

/**USE OF ELSE-IF LADDER**/
#include <stdio.h>
int main()
{
    int marks; char grade;
    printf("\n\nEnter average marks of students:");

```

```

scanf("%d",&marks);
if(marks > 79)
{
    grade = 'A';
    else if(marks > 59)
    grade = 'B';
    else if(marks > 49)
    grade = 'C';
    else if(marks > 39)
    grade = 'D';
    else
    grade = 'F';
    printf("\nGrade of the student is %c",grade);
}
return 0;
}

```

(D) SWITCH STATEMENT

The **nested if** allows selecting one of the many alternatives but it is time consuming because it tests all the conditions and based on the result a particular branch is taken for execution.

To overcome this disadvantage, the **switch** statement is used.

The **switch** statement provides a *multiple way branching*.

- It allows the user to select any one of the several alternatives, depending on the value of the expression.

Syntax:

```

switch (expression)
{
case value1: block1;
            break;
case value2: block2;
            break;
case value3: block3;

```



```

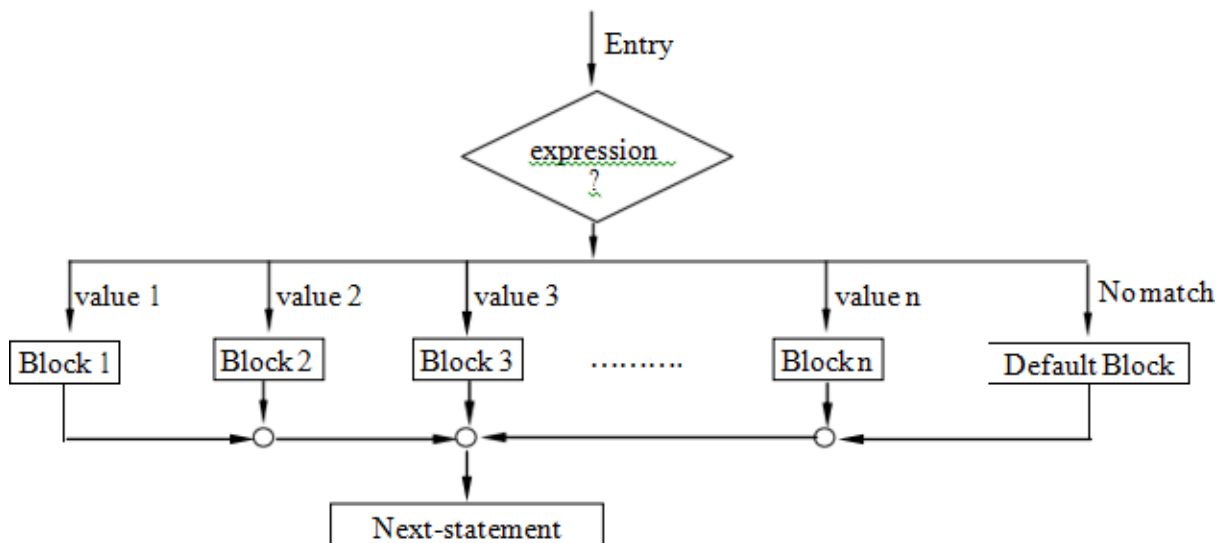
        break;
        -----
        -----
        -----
case value n:block n;
        break;
default:default block;
        break;
}

```

Note:

- (1) If a break statement is omitted in switch statement, all the next cases until the very next break statement will be executed.
- (2) Break statement after the default case need not be included, because, any how the control will be transferred to the very next statement after switch statement.

Flowchart:



Example 1:

```
/**USE OF SWITCH STATEMENT***/
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int i;
```

```
printf("\nEnter i (1 to 4):");
```

```
scanf("%d",&i);
```

```
switch(i)
```

```
{
```

```
case 1:
```

```
printf("\nI am in case 1...");
```

```
break;
```

```
case 2:
```

```
printf("\nI am in case 2...");
```

```
break;
```

```
case 3:
```

```
printf("\nI am in case 3...");
```

```
break;
```

```
case 4:
```

```
printf("\nI am in case 4...");
```

```
break;
```

```
default:
```

```
printf("\nI am in default case...");
```

```
}
```

```
return 0;
```

```
}
```

Example 2:

```

/**USE OF SWITCH STATEMENT***/
#include <stdio.h>
int main()
{
float len,br,ht,base,side,radius;
char shape;
printf("\n\nEnter your choice:");
printf("\nEnter 'c' to compute area of circle:");
printf("\nEnter 's' to compute area of square:");
printf("\nEnter 'r' to compute area of rectangle:");
printf("\nEnter 't' to compute area of triangle:");
scanf("%c",&shape);
switch(shape)
{
case 'c':
case 'C':
printf("\n\nArea of a Circle...");
printf("\nEnter the radius of the circle:");
scanf("%f",&radius);
area = 3.142*radius*radius;
printf("\n\nArea of a Circle is %f",area);
break;

case 's':
case 'S':
printf("\n\nArea of a Square...");
printf("\nEnter the side of the square:");
scanf("%f",&side);
area = side*side;
printf("\n\nArea of a Square is %f",area);
break;

case 'r':
case 'R':
printf("\n\nArea of a Rectangle...");
printf("\nEnter length and breadth of the rectangle:");
scanf("%f%f",&len,&br);
area = len*br;
printf("\n\nArea of Rectangle is %f",area);
break;

```

```

case 't':
case 'T':
printf("\n\nArea of a Triangle...");
printf("\nEnter base and altitude of the triangle:");
scanf("%f %f",&base,&ht);
area = 0.5*base*ht;
printf("\n\nArea of Triangle is %f",area);
break;

default:
printf("\n\nSorry...Unable to identify your choice");
}
return 0;
}

```

Example 3: Write a program in C using switch statement to device a Basic Arithmetic Calculator.

```

/****USE OF SWITCH STATEMENT****/

#include <stdio.h>

int main()
{
float n1,n2; char oper;
printf("\n\nEnter the two numbers which must be operated:");
scanf("%f%f",&n1,&n2);
printf("\n\n Select the operation you want to perform:");
printf("\nEnter 'A' to compute SUM:");
printf("\nEnter 'S' to compute DIFFERENCE:");
printf("\nEnter 'M' to compute PRODUCT:");
printf("\nEnter 'D' to compute QUOTIENT:");
scanf("%c",&oper);
switch(oper)
{
case 'a':
case 'A':
printf("\n\nPerforming addition...");

```

```

printf("\n\nThe sum of %f & %f is %f",n1,n2,n1+n2);
break;

case 's':
case 'S':
printf("\n\nPerforming subtraction...");
printf("\n\nThe difference of %f & %f is %f",n1,n2,n1-n2);
break;

case 'm':
case 'M':
printf("\n\nPerforming multiplication...");
printf("\n\nThe product of %f & %f is %f",n1,n2,n1*n2);
break;

case 'd':
case 'D':
printf("\n\nPerforming division...");
printf("\n\nThe quotient of %f & %f is %f",n1,n2,n1/n2);
break;

default:
printf("\n\nSorry...Unable to identify your choice");
}
return 0;
}

```

UNCONDITIONAL BRANCHING:

THE GOTO STATEMENT

The conditional control statements control the flow of execution of statements based on the value of the conditional expression.

- The **goto** statement is an unconditional statement in C which allows the flow of execution of statements to be altered without the use of any conditional

expression.

- The **goto** statement transfers the control from one point to another in a C program. This is also known as *Jumping of control* or *Unconditional Jumping*.

The syntax of **goto** statement is as follows:

goto label;

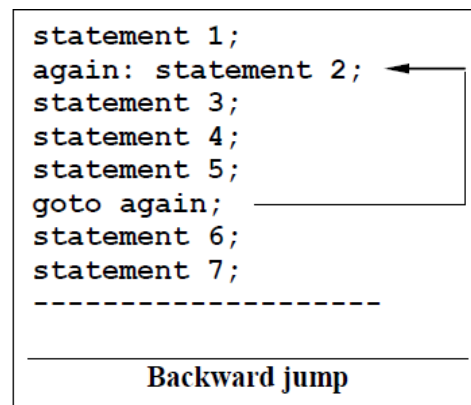
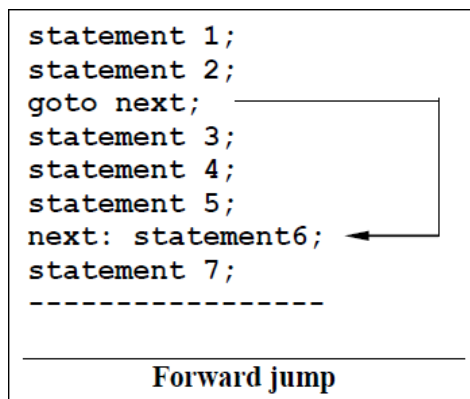
where, **goto** is a keyword, and

label is a symbolic constant or an identifier which need not be pre-declared.

- Two types of jumping (branching) is possible by using **goto** statement:

(1) Forward jump: Here the statements immediately following the **goto** statement will be skipped and control is transferred to the statement beginning with the symbolic constant **lable**.

(2) Backward jump: Here the symbolic constant **lable** is placed before the **goto** statement and the statements between the **lable** and **goto** statements are repeated resulting in looping.



Example 1: Write a program in C using goto statement to read 5 numbers and compute their sum.

```
/**USE OF GOTO STATEMENT***/  
#include <stdio.h>  
int main()  
{  
int count = 0;  
float sum = 0.0, num;
```

```

printf("\n\nEnter five numbers:");
read: scanf("%f",&num);
    sum += num;
    count++;
    if (count < 5) ? expression1 : expression2
    goto read;
    printf("\n    The sum of 5 numbers is %f", sum);
return 0;
}

```

Example 2: Write a program in C using goto statement to accept the number and reverses it.

```

#include <stdio.h>
int main()
{
    int number, rev = 0, digit, temp_num;
    printf("\nEnter a number:\n");
    scanf("%d",&number);
    temp_num = number;
    START:
        digit = number%10;
        rev = rev*10+digit;
        number /= 10;
        if (number > 0)
            goto START;
    printf("\nInput number = %d\n", temp_num);
    printf("\nReversed number = %d\n", rev);
    return 0;
}

```

The ?: operator

The ?: operator is popularly known as *conditional operator*, useful for making *two-way* decisions and takes *three* operands.

The general form of the use of conditional operator is as follows:

Conditional_expression

- The **conditional_expression** is evaluated first. If the result is true (nonzero),

expression1 is evaluated and is returned as the value of the conditional expression.
Otherwise, **expression2** is evaluated and its value is returned.

Example 1:

$$y = 1.5x + 3 \quad \text{if } x > 2$$

This can be evaluated using the conditional operator as:

$$y = (x > 2) ? (2 * x + 5) : (1.5 * x + 3);$$

- The conditional operator may be *nested* for evaluating more complex assignment decisions.

Example 2:

for $x \leq 2$		
Salary = {	$4x + 100$	for $x < 40$
	300	for $x = 40$
	$4.5x + 150$	for $x > 40$

This complex equation can be written using the conditional operator as:

$$\text{Salary} = (x \neq 40) ? ((x < 40) ? (4 * x + 100) : (4.5 * x + 150)) : 300$$

Example 3:

int a = 1, b = 2, ans;

```

if (a == 1)
{
    if (b == 2)
    {
        ans = 3;
    } else {
        ans = 5;
    }
} else {
    ans = 0;
}

```



```
printf ("%d\n", ans);
```

Using Conditional Operator, the above code can be written as:

```
int a = 1, b = 2, ans;  
ans = (a == 1 ? (b == 2 ? 3 : 5) : 0);  
printf ("%d\n", ans);
```

Output: 3

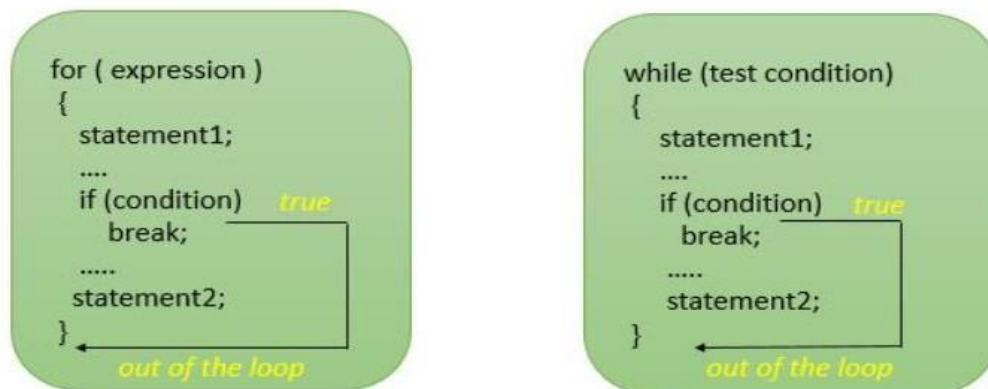
THE BREAK STATEMENT

In any loop break is used to jump out of loop skipping the code below it without caring about the test condition.

It interrupts the flow of the program by breaking the loop and continues the execution of code which is outside the loop.

The common use of break statement is in switch case where it is used to skip remaining part of the code.

Structure of Break statement



Example: C program to take input from the user until he/she enters zero.

```
#include <stdio.h>  
int main ()
```

```

{
int a;
while (1)
{
printf("enter the number:");
scanf("%d", &a);
if ( a == 0 )
break;
}
return 0;
}

```

In above program, while is an infinite loop which will be repeated forever and there is no exit from the loop.

So the program will ask for input repeatedly until the user will input 0.

When the user enters zero, the if condition will be true and the compiler will encounter the break statement which will cause the flow of execution to jump out of the loop.

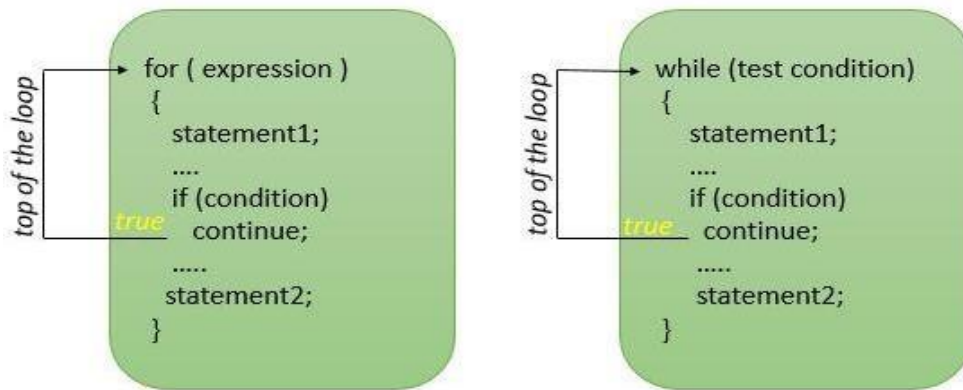
THE CONTINUE STATEMENT

Like a break statement, continue statement is also used with if condition inside the loop to alter the flow of control.

When used in while, for or do...while loop, it skips the remaining statements in the body of that loop and performs the next iteration of the loop.

Unlike break statement, continue statement when encountered doesn't terminate the loop, rather interrupts a particular iteration.

Structure of Continue statement



Example: C program to print sum of odd numbers between 0 and 10

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
int a,sum = 0;
```

```
for (a = 0; a < 10; a++)
```

```
{
```

```
    if ( a % 2 == 0 )
```

```
        continue;
```

```
    sum = sum + a;
```

```
}
```

```
printf("sum = %d",sum);
```

```
return 0;
```

```
}
```

Output

sum = 25

Chapter 6: Decision making and Looping

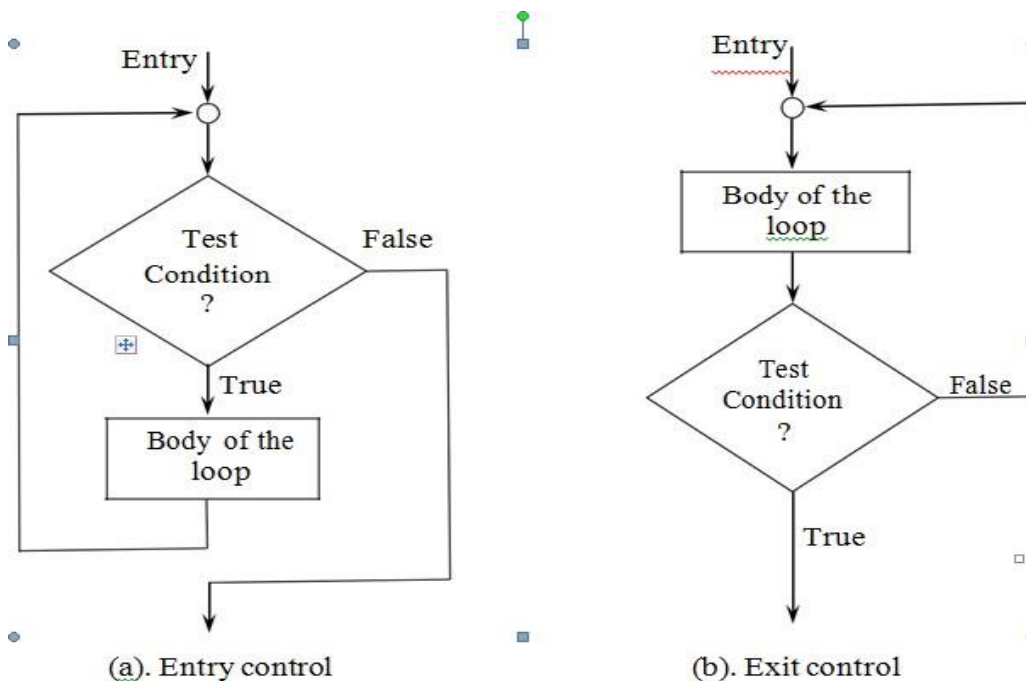
Looping is a mechanism by means of which a set of statements are executed repeatedly for a fixed number of times or until a condition is fulfilled.

Looping is also called **repetitive** or **iterative** control mechanism. A program loop consists of **two** segments.

- Body of the loop.
- Control statement.
-

Control statement tests certain conditions and directs the repeated execution of the statements contained in the *body of the loop*.

Depending on the position of the *control statement* in the loop, a control structure is classified as **entry – controlled loop** and **exit – controlled loop**.



Loop control structures

- In **entry – controlled loop**, the control conditions are tested before the start of the loop execution. If the conditions are not satisfied, then the body of the loop will not be executed.
- In **exit – controlled loop**, the test is performed at the end of the loop and therefore the body is executed unconditionally for the first time.

Steps of efficient looping mechanism

- **Initialization:** To set the initial value for the loop counter; it may be an increment or decrement counter.
- **Decision:** An appropriate **test_condition** to determine whether the loop should be executed or not and eventually should allow the loop to terminate. Otherwise it may form an *infinite loop*.
- **Body of the loop:** To execute the body of the loop when the *test condition* is TRUE.
- **Updating loop counter:** Increment or decrementing the loop counter.

C language provides **three** types of loop structures.

- **while** statement.
- **do while** statement.
- **for** statement.

(A) THE WHILE STATEMENT

- It executes a set of statements repeatedly as long as the specified condition is true.

Syntax of the while statement:

<pre><u>while</u> (<u>test condition</u>) <u>statement</u>; <u>statement</u> x; <u>statement</u> y;</pre>
'while' with single statement

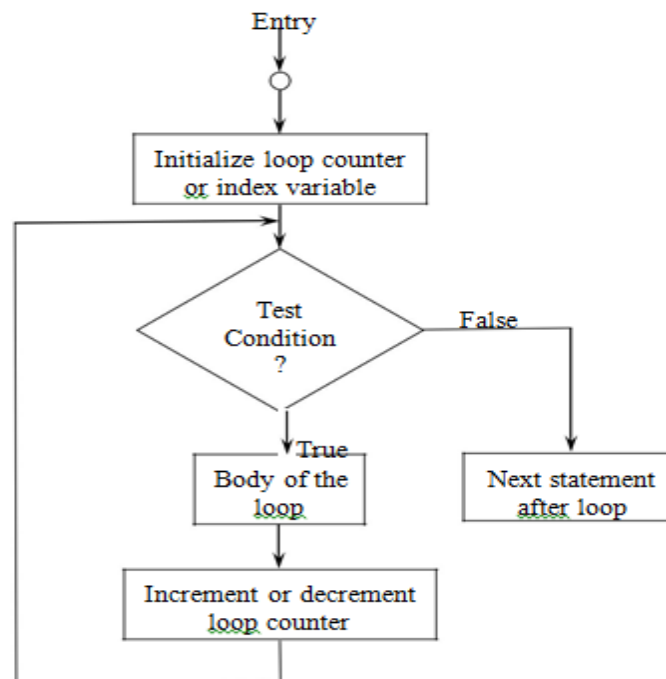
while is a keyword.

test condition is a logical or relational expression that results in either TRUE or FALSE.

<pre><u>while</u> (<u>test condition</u>) { <u>statement1</u>; <u>statement2</u>; <u>statement3</u>; } <u>Statement</u> x; <u>Statement</u> y;</pre>
'while' with compound statement

- The **while** is an *entry – controlled loop* statement.
- The *test condition* is evaluated and if the condition is *true*, then the body of the loop is *repeatedly* executed.
- If the condition is *false*, then control comes out of the loop and continues with next executable statement.
- The value of the *test condition*, will change during every pass of while loop. If it is not changed, then while loop enters an infinite loop (i.e. control does not come out).

Flowchart:



Example (1): Using while statement, write a C program to accept your name and prints it desired number of times.

```
/**USE OF while STATEMENT**/  
#include <stdio.h>  
  
int main()  
{  
    char name[20];  
    int ktimes, i;  
    printf("\nWhat is your name?\n");  
    gets(name);  
    printf("\nHow many times to print your name\n");  
    scanf("%d",&ktimes);  
    i=0;  
  
    while(i < ktimes)
```

```

{
    puts(name);
    i++;
}
return 0;
}

```

Output of program:

```

What is your name?

NMAMIT

How many times to print your name 3

NMAMIT
NMAMIT
NMAMIT

```

Example (2): Using while statement, write a C program to compute **x** to the power **n**. where n is non negative number.

```

/**USE OF while STATEMENT***/
#include <stdio.h>
int main()
{
    int count, n;
    float x,y;

    printf("\nEnter the values of x and n:");
    scanf("%f %d",&x, &n);
    y=1.0;

    count=1;

```



```

while(count <= n)
{
    y = y*x;
    count++;
}
printf("x = %f; n = %d; x to the power n = %f", x, n, y);
return 0;
}

```

Output of program:

Enter the values of x and n: 2.5 4

x = 2.500000; n = 4; x to the power n = 39.062500

Enter the values of x and n: 0.5 4

x = 0.500000; n = 4; x to the power n = 0.062500

(B) THE DO WHILE STATEMENT

Syntax of the do...while statement:

```

do
{
    statement1;
    statement2;
}
while (test_condition);
Statement x;
Statement y;

```

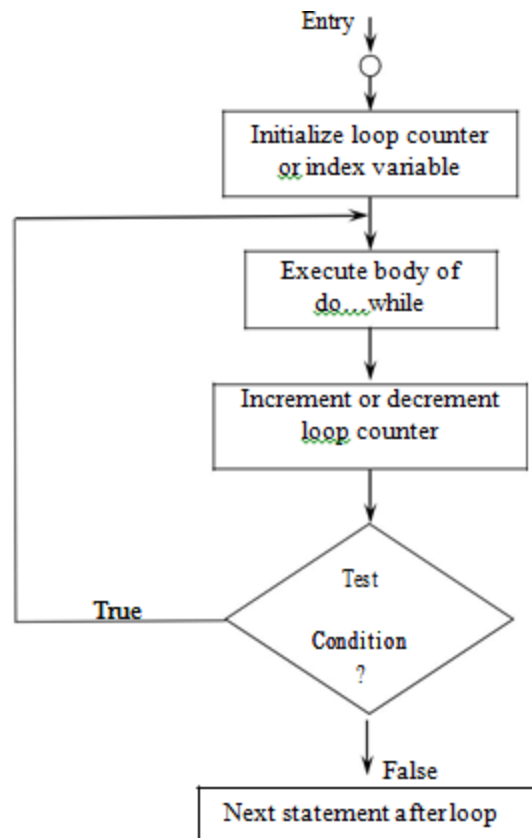
do and **while** are keywords.

test_condition is a logical or relational expression that results in either TRUE or FALSE.

- On reaching the **do** statement, the program proceeds to evaluate the body of the loop first. At the end of the loop, the test_condition in the **while** statement is evaluated.
- If the condition is *true*, the program continues to evaluate the body of the *loop* once again. This process continues as long as the *condition* is true.

- When the condition becomes *false*, the *loop* will be terminated and the control goes to the statement that appears immediately after the while statement.
- Since the test_condition is evaluated at the bottom of the loop, the **do...while** construct provides an **exit controlled** loop and therefore the body of the loop is always executed at least once.

Flowchart:



Example (1): Using do..while, write a C program to print the sum of all integers between 1 and 50.

```

/**USE OF do...while STATEMENT***/
#include <stdio.h>
int main()
{
    int odnum, sum = 0;

```

```

odnum = 1;

do
{

    sum += odnum;

    odnum += 2;

}

while(odnum <= 50);

printf("Sum = %d\n", sum);

return 0;

}

```

Example (2): Using do...while statement, write a C program to print the multiplication table from 1 x 1 to 12 x 10 as shown below

```

1  2    3 . . 10
2  4    6 . . 20
3  6    9 . . 30
.  . . . .
.  . . . .
12 .    . . . 120

```

```

/**USE OF do...while STATEMENT***/
#include <stdio.h>

#define COLMAX 10
#define ROWMAX 12
int main()

{

```

```

int row, column, y;
row = 1;

printf("          MULTIPLICATION TABLE          \n");
printf(".....\n");
/*..OUTER LOOP BEGINS..*/
do
{
    column = 1;
    /*..INNER LOOP BEGINS..*/
    do
    {
        y = row * column;
        printf("%d", y);
        column = column + 1;

        } while(column <= COLMAX);
    /*..INNER LOOP ENDS..*/
    printf("\n");
    row = row + 1;
} while(row <= ROWMAX);
/*..OUTER LOOP ENDS..*/
printf(".....\n");
return 0;
}

```

(C) THE FOR STATEMENT

The **for** is an *entry – controlled loop* statement. This statement is used when the programmer knows how many times a set of statements are executed.

Syntax of the *simple for* statement:

```
for(initialization; test_condition; increment)
{
    statement1;
    statement2;
}
Statement x;
Statement y;
```

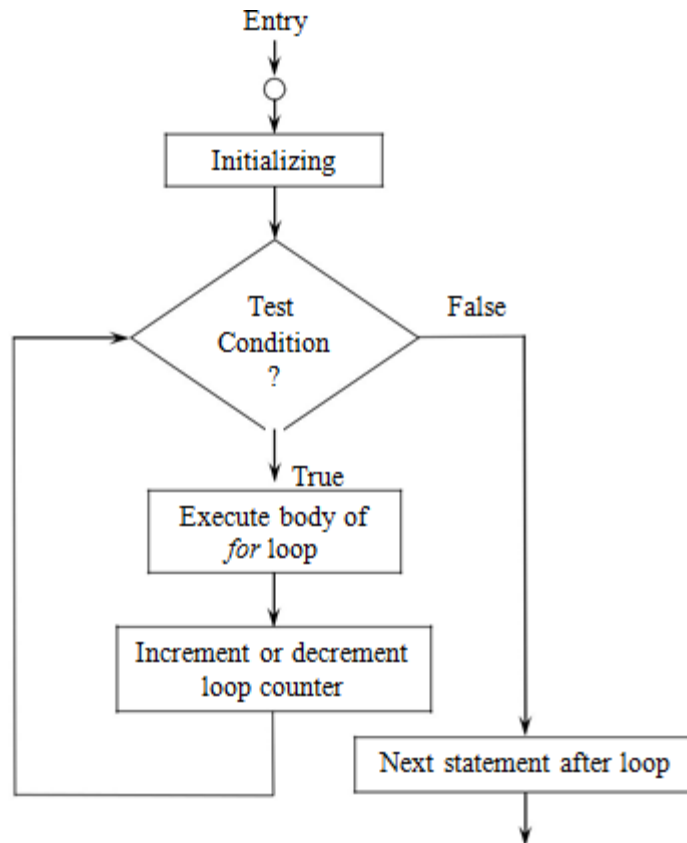
The execution of the **for** statement is as follows:

- **Initialisation** of the *control variable* is done first. Using assignment statements such as $i = 1$ and $count = 0$. The variables i and $count$ are known as *loop – control variables*.
- The value of the *control variable* is tested using the **test_condition**. The *test_condition* is a relational expression, such as $i < 10$ that determines when the loop will exit. If the condition is *true*, the body of the loop is executed; otherwise the loop is terminated and the execution continues with the statement the immediately follows the loop.
- When the body of the loop is executed, the control is transferred back to the **for** statement after evaluating the last statement in the loop. Now, the *control variable* is **incremented** using an assignment statement such as $i = i + 1$ and the new value of the *control variable* is again tested to see whether it satisfies the loop condition. If the condition is satisfied, the body of the loop is again executed. This process continues till the value of the control variable fails to satisfy the *test_condition*.
- The **for** statement allows for *negative increment* also.
- The three sections enclosed within parentheses must be separated by **semicolons**. There is **no semicolon** at the end of the **increment** section as well as after the parentheses.
- *Braces* are optional when the body of the loop contains only *one* statement.

The number of *iterations* required to execute the body of **for** is computed using the formula:

Number of iterations = (Final value – initial value + step increment) / step increment

Flowchart:



Example (1): Using *for* statement, write a C program to print the sum of first 50 natural numbers.

/**USE OF for STATEMENT***/

```
int main()
{
    for(num=1; num<=50; num++)
    {
        sum = sum + num;
    }
}
```

```

printf("Sum = %d\n", sum);
return 0;
}

```

Example (2): Using *for* statement, write a C program to print factorial of a given number.

Hint: The formula to compute factorial of a number **n** is

$$n! = n * (n-1) * (n-2) * (n-3) * * 3 * 2 * 1$$

```

#include <stdio.h>

int main()
{
    int c, n, f = 1;

    printf("Enter a number to calculate its factorial\n");
    scanf("%d", &n);
    for (c = 1; c <= n; c++)
        f = f * c;
    printf("Factorial of %d = %d\n", n, f);
    return 0;
}

```

Example (3): Using *for* statement, write a C program to generate the first n Fibonacci numbers. In fibonacci sequence each number is generated by adding the previous two numbers. Assuming the first two numbers as 0 and 1. Then the series,

0 1 1 2 3 5 8 13 ...

```

/**Program to generate the first n fibonacci numbers***/
#include <stdio.h>
int main()
{
    int n, n1, n2, n3;

```

```

printf("\n Enter the limits of fibonacci series:\n");
scanf("%d", &n);
printf("\n Enter the first & second numbers in fibonacci series:\n");
scanf("%d %d", &n1, &n2);
printf("\n%d %d", n1, n2);
for(i = 3; i <= n; i++)
{
    n3 = n1 + n2;
    printf("\t%d", n3);
    n1=n2;
    n2=n3;
}
return 0;
}

```

Additional features of for loop:

- More than *one variable* can be **initialized** at a time in the **for** statement, separated by a **comma operator**.

for(p=1, n=0; n < 17; ++n)

- The **increment** section may also have more than one part.

for(n=1, m=50; n <= m; n++, m--)

- The **test_condition** may have *any compound relation* and the testing need not be limited only to the loop control variable.

```

sum = 0;
for(i=1; i < 20 && sum < 100; ++i)
{
    sum = sum + i;
    printf("%d", sum);
}

```


- It is also permissible to use *expressions* in the *assignment* statements of **initialization** and **increment** sections.

```
for(x = (m + n)/2; x > 0; x = x/2)
```

- In **for** statement one or more sections can be omitted, if necessary. In such cases, the sections are left blank. However, the *semicolon* separating the sections must remain.

```
m = 5;
for(; m != 100;)
{
    printf("%d", sum);
    m = m + 5;
}
```

- If the **test_condition** is not present, the **for** statement sets up an *infinite loop*. Such loops can be broken using **break** or **goto** statements in the loop.
- We can set up *time delay loops* using the *null* statement as follows: This loop is executed 1000 times without producing any output; it simply causes a time delay. The *body of the loop* contains *only a semicolon*, known as *null* statement.

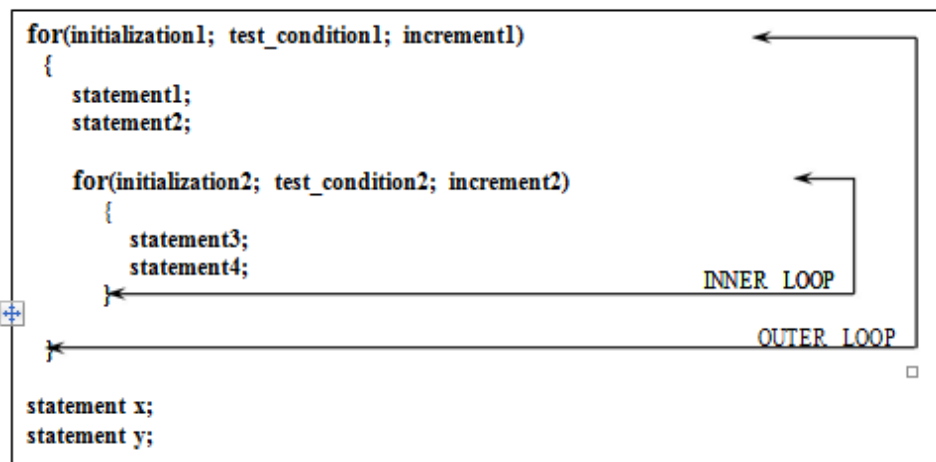
```
for(j = 1000; j > 0; j = j - 1)
;
```

- This can also be written as **for(j = 1000; j > 0; j = j - 1);** This implies that the C compiler will not give an error message if we place a *semicolon* by mistake at the end of a **for** statement. The *semicolon* will be considered as a *null statement* and the program may produce *some nonsense*.

Nesting of for loops:

- If one **for** statement is completely placed within the other, it is known as **nested for statement**.
- Each **for** statement must have different index variable.
- Nesting may be up to 15 levels in ANSI C; many compilers allow more.

Syntax of the *nested* for statement:



Example:

A class of **n** students take an annual examination in **m** subjects. Write a C program to read the marks obtained by each student in various subjects and to compute and print the total marks obtained by each of them.

```
/**ILLUSTRATION OF NESTED LOOPS***/
#include <stdio.h>
#define FIRST 360
#define SECOND 240
int main()
{
    int i, j, n, m, roll_number, marks, total; clrscr ();
    printf("Enter the number of students and subjects\n");
    scanf("%d %d", &n, &m);
    printf("\n");

    for(i = 1; i <= n; ++i)
    {
        printf("Enter Roll Number:");
        scanf("%d", &roll_number);
        total = 0;

        printf("\nEnter marks of %d subjects for ROLL NO %d \n", m, roll_number);
        for(j = 1; j <= m; j++)
        {
            scanf("%d", &marks);
            total = total + marks;
        }
        printf("TOTAL MARKS = %d", total);

        if (total >= FIRST) printf("(First Division)\n\n");
        else if (total >= SECOND)
            printf("(Second Division)\n\n");
        else
            printf("*** FAIL ***\n\n");
    }
}
```

```
return 0;
}
```

Example: Write a C program to generate the following number pattern

```
      1
    2 3 2
  3 4 5 4 3
4 5 6 7 6 5 4
5 6 7 8 9 8 7 6 5
```

and so on.

```
#include <stdio.h>
int main()
{
    int i, j, k=1, n, p;
    printf("Enter the number of rows\n");
    scanf("%d", &n);
    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= (n-i); j++)
        {
            printf(" ");
        }
        for(j = 1; j <= i; j++)
        {
            printf("%d", k++);
        }
        for(j = (i-1); j >= 1; j--)
        {
```

```

        p = k - 2;
        k = k - 1;

        printf("%d", p);
    }
    printf("\n");
}
return 0;
}

```

JUMPS IN LOOPS

- When executing the body of loop, sometimes it becomes desirable to skip a part of the loop or to leave the loop as soon as a certain condition occurs. *For example, in the case of searching for a particular name in a list containing, say, 100 names, program loop must be terminated as soon as the desired name is found.*
- C permits a jump from one statement to another *within a loop* as well as a *jump out of a loop*.

Jumping Out of a loop:

- An early exit from a loop can be accomplished by using the **break** statement or the **goto** statement.
- **break** statement or the **goto** statement can be used within if ...else, switch, while, do ...while, or for loops.
- When the **break** statement is encountered *inside the loop*, the loop is immediately exited and program continues with the statement immediately following the loop.
- When the loops are *nested*, the **break** would only exit from the loop containing it. That is **break** will *exit only a single loop*.

- Since a **goto** statement can transfer the control to any place in the program, it is useful to provide *branching* within a loop.
- Important use of **goto** is to exit from deeply nested loops when an error occurs. A simple **break** statement would not work here.
- But it is good practice to *avoid* using **goto** statement, because many compilers generate *less efficient code* if **goto** is used, and also using many **goto** statements makes *program logic complicated* and renders the program unreadable.

Example (1): Write a C program to find whether given number is prime number.

```

/**Program to find if a given number is prime***/
#include <stdio.h>
int main()
{
    int i, num, flag=0;
    printf("\n Enter the number to be checked as prime:\n");
    scanf("%d", &num);

    for(i = 2; i <= (num - 1); i++)
    {
        if (num % i == 0)
        {
            flag = 1;
            break;
        }

        if (flag != 1)
            printf("\n Number Entered %d is a Prime", num);

        else

            printf("\n Number Entered %d is not a Prime", num);
    }
    return 0;
}

```

Output of program:

(1) Enter the number to be checked as prime: 3

Number Entered 3 is a Prime

(2) Enter the number to be checked as prime: 10

Number Entered 10 is not a Prime

Example (2): Write a C program to list prime numbers between 1 & 300.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, num, flag, np = 0;
```

```
    printf("\n List of Prime numbers between 1 & 300\n");
```

```
    for(num = 1; num <= 300; num++)
```

```
    {
```

```
        flag = 0;
```

```
        for(i = 2; i <= (num - 1); i++)
```

```
        {
```

```
            if ((num % i) == 0)
```

```
            {
```

```
                flag = 1;
```

```
                break;
```

```
            }
```

```
        if (flag != 1)
```

```
        {
```

```
            printf("%3d \t", num);
```

```
        }
```

```
        np = np + 1;
```

```
    }
```

```
    printf("\n Number of Prime numbers between 1 & 300 = %d", np);
```

```
    }  
    return 0;  
}
```

Skipping a part of a loop:

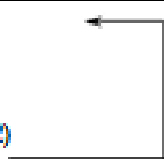
- To skip certain statements in the loop and to continue with the next iteration of the loop **continue** statement is used.
- **continue** statement is used as a bypasser.
- Control does not come out of the loop instead skips the remaining statements within the body of that loop and transfer to the beginning of the loop.

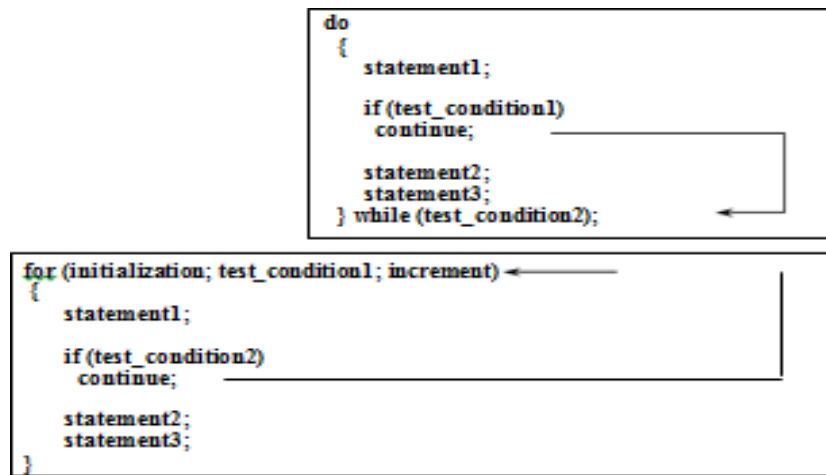
Syntax of the continue statement:

```
statement1;  
statement2;  
  
continue;  
  
statement3;  
statement4;
```

Use of the continue statement in loops:

```
while (test_condition1)  
{  
    statement1;  
    if (test_condition2)  
        continue;  
    statement2;  
    statement3;  
}
```

A diagram illustrating the effect of the 'continue' statement. It shows a rectangular loop structure. An arrow starts from the 'continue;' line, goes to the right, then turns upwards, and finally turns left to point back at the 'while (test_condition1)' line, indicating that the loop restarts from the beginning, skipping the statements between 'continue;' and the next iteration.



- In **while** and **do** loops, **continue** causes the control to go directly to the **test_condition** and then to continue the iteration process.
- In **for** loop, the **increment** section of the loop is executed *before* the **test_condition** is valuated.

Example: Write a C program to evaluate the square root of series of numbers and print the results. The process should stop when the number 9999 is typed. Use continue statement to skip, if *negative* number is entered.

```
/**USE OF continue STATEMENT***/
#include <stdio.h>
#include <math.h>
main()
{
    int count, negative;
    double number, sqrt;

    printf("Enter 9999 to stop\n");
    count = 0;
```

```

negative = 0;

while(count <= 100)
{
    printf("Enter a number:");
    scanf("%lf", &number);
    if (number == 9999)
        break;
    if (number < 0)
    {
        printf("\n Number is negative\n\n");
        negative++;
        continue;
    }

    sqroot = sqrt(number);
    printf("\n Number\t\t =%lf \n Square root\t =%lf \n\n", number, sqroot);
    count++;
}
printf("\n Number of items done = %d\n", count);
printf("\n\n Negative items = %d\n", negative);
printf("END OF DATE\n");
return 0;
}

```

Output of program:

```
Enter 9999 to stop
Enter a number: 25.0
Number          = 25.000000
Square root     = 5.000000

Enter a number: 40.5
Number          = 40.500000
Square root     = 6.363961

Enter a number: -9
Number is negative

Enter a number: 16
Number          = 16.000000
Square root     = 4.000000

Enter a number: -14.75
Number is negative

Enter a number: 80
Number          = 80.000000
Square root     = 8.944272

Enter a number: 9999

Number of items  done = 4
Negative items   = 2
END OF DATA
```

Additional Info:

Concise Test expressions:

- We often use **test expressions** in the **if**, **for**, **while** and **do** statements that are *evaluated* and compared with zero for making branching decisions. Since every *integer expression* has a *true/false* value, we need not make explicit comparisons with zero.
- For instance, the *expression x* is **true** whenever *x is not zero*, and **false** when *x is zero*. Applying **! operator**, we can write concise test expressions without using any relational operators.

if (*expression* == 0)

is equivalent to

if (*!expression*)

Similarly,

if (*expression* != 0)

is equivalent to

if (*expression*)

Example:

if (m % 5 == 0 && n % 5 == 0)

is same as

if (!(m % 5) && !(n % 5))

Chapter 7: Arrays

Need for an Array:

Many applications require the processing of multiple data items that have common characteristics (e.g., set of numbers, set of names).

Array is a derived data type which is used to store similar data items in contiguous memory locations under a single variable name. In other words, **array** is a *group of data elements* of same data type that share a *common name*.

An array holds a fixed number of equally sized data elements of the same type. The individual elements are accessed by specifying the index number called subscript. Subscript specifies position of data item in an array, and subscript must be an unsigned positive integer. Hence arrays are also called **subscripted variables**.

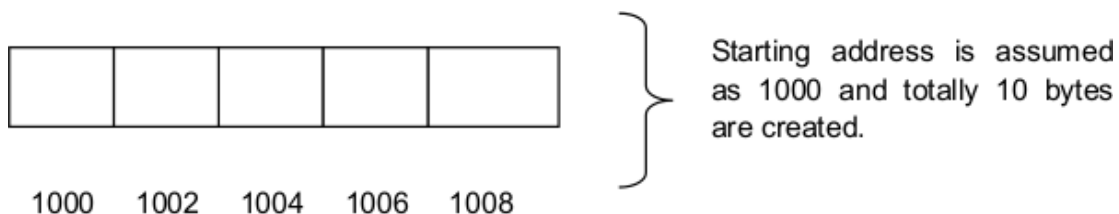
Types of Arrays:

1. One dimensional array
2. Multi dimensional arrays (2-D, 3-D, etc.)

Memory Organization of an Array:

The elements in an array are always stored in consecutive memory locations. If an array of 5 integers elements is created, totally 10 contiguous bytes will be allocated in memory.

Note: size of an integer is assumed to be 2 bytes



Individual memory location is referred by index. [index 0 refers first location , index 1 refers second location, etc.].

Address of an array element is calculated as below:

Address of i th location = base address + (size of the individual data element * index i)

Address of 0 th element = $1000 + (2 * 0) = 1000$

Address of 1 st element = $1000 + (2 * 1) = 1002$ and so on.

In C, the name of the array refers to the base address of the array.

DECLARATION OF 1D ARRAY

General Form:

datatype arrayname[size] ;

This is called a single-dimensional array. The size must be an integer constant greater than zero and datatype can be any valid C data type.

For example, to declare a 10-element array called balance of type double, use this statement –

`double balance[10];`

Here balance is a variable array which is sufficient to hold up to 10 double numbers.

Arrays are defined by appending an integer encapsulated in square brackets at the end of a variable name. Each additional set of brackets defines an additional dimension to the array (multi-dimensional arrays). When addressing an element in an array, indexing begins at 0 and ends at one less than the defined size of an array.

Example:

`int x[5];`

defines an integer array x of 5 integers, starting at x[0], and ending at x[4].

`char str[16]="qwerty";`

defines a character array, which represents a string of maximum of 16 characters.

`float sales_amt[10];`

defines a floating point array sales_amt of 10 floating point numbers, starting at sales_amt[0] and

ending at sales_amt[9].

2D array:

```
int matrix[2][2];
```

defines a 2*2 matrix (totally 4 elements) of integers.

INITIALIZING 1D ARRAYS (COMPILE-TIME)

General Forms:

1. **datatype arrayname[size] = {value(s)};**
2. **datatype arrayname[] = {value(s)};**

The first one initializes an array with size, that is, number of elements to be stored. The second one without size and it depends on the number of elements given.

Thus we can initialize an array using a single statement in a declaration part of the program as follows –

```
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

The number of values between braces { } cannot be larger than the number of elements that we declare for the array between square brackets [].

If you omit the size of the array, an array just big enough to hold the initialization is created. Therefore, if you write –

```
double balance[] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

You will create exactly the same array as you did in the previous example.

Following is an example to assign a single element of the array:

```
balance[4] = 50.0;
```

The above statement assigns the 5th element in the array with a value of 50.0. All arrays have 0 as the index of their first element which is also called the base index and the last index of an array will be total size of the array minus 1.

Shown below is the pictorial representation of the array we discussed above –

	0	1	2	3	4
balance	1000.0	2.0	3.4	7.0	50.0

Some more examples:

```
int a[5]={1,2,3,4,5}; /* a[0] = 1, a[1] = 2 , a[2] = 3 ,a[3] = 4 and a[4] = 5*/
int a[5]={0};          /* all the array elements are initialized to zero*/
int a[5]={1,2,3,4};    /*a[4] = 0*/
int a[ ] = {1,2,3,4}; /* a[0]=1, a[1]=2, a[2]=3, a[3]=4
                    (if size not specified, size depends upon the number of values initialized. ) */
float b[2]={10.2,45.34}; /* b[0] = 10.20 , b[1] = 45.34 */
```

ACCESSING ARRAY ELEMENTS

The array elements are accessed by specifying the subscript or index.

General Form:

arrayname[index or subscript]

Example:

x[0] to access the 1st element in array
 x[4] to access the 5th element in array
 str[2] to access the 3rd character in the string (character array)
 sales_amt [8] to access the 9th sales amount in the array

Basic Operations on Arrays:

Basic operations allowed on arrays are storing, retrieving, and processing of array elements. Insertion and deletion can be done which consume more time.

GETTING THE VALUE FOR ARRAYS (RUN-TIME INITIALIZATION):

Input statement is used to get the values for an array.

Example:

- ```
int a[3];
```
- (1) `scanf("%d", &a[0]); /* gets value for 1st location*/`  
`scanf("%d", &a[1]); /* gets value for 2nd location*/`  
`scanf("%d", &a[2]); /* gets value for 3rd location*/`
  - (2) `scanf("%d%d%d", a, a+1, a+2); /* gets value for first 3 locations array name`  
`has the base address pointer)*/`
  - (3) Usually loop statement is used to input the array elements.  
`for(i=0;i<3;i++)`  
`scanf("%d",&a[i]);`

## **PRINTING OUT THE ARRAY ELEMENTS**

### Example:

- ```
int a[3];
```
- (1) `printf("%d", a[0]); /*prints value of 1st location*/`
`printf("%d",a[1]); /*prints value of 2nd location*/`
`printf("%d", a[2]); /*prints value of 3rd location*/`
 - (2) `printf("%d%d%d",a[0],a[1],a[2]); /* prints value of first 3 locations*/`
 - (3) `for(i=0;i<3;i++) /* loop statement is used to print the array elements */`
`printf("%d",a[i]);`

Multi-dimensional Arrays:

The elements of an array can themselves be arrays. Two dimensional arrays can be viewed as group of one dimensional array (rows & columns) and 3 dimensional arrays can be viewed as set of two dimensional arrays and so on. Multidimensional arrays will also occupy the contiguous memory locations.

TWO-DIMENSIONAL ARRAY – DECLARATION

Two-dimensional arrays are defined in the same way as one-dimensional array, except that a separate pair of square brackets are required for second dimension.

General Form:

datatype arrayname[row][column];

Example:

Create a 2-D array with 2 rows and 2 columns

```
int a[2][2]; /*Allocates 8 bytes of contiguous  
memory locations. (2*2=4 elements) */
```

Elements are stored in row major order. Elements of 1st row are stored first and then the elements of next row. It is necessary to specify the size of the column in declaration.

Assume that array starts at location 1000,

a[0][0] will be in location 1000 - row 0 & column 0

a[0][1] will be in location 1002 - row 0 & column 1

a[1][0] will be in location 1004 - row 1 & column 0

a[1][1] will be in location 1006 - row 1 & column 1

TWO-DIMENSIONAL ARRAY INITIALIZATION (COMPILE-TIME)

Two-dimensional arrays can also be initialized in the declaration statement. In partial initialization, the uninitialized array elements are initialized to Zero.

Example:

```
int num[2][3] = {1,2,3,4,5,6};
```

```
int num[2][3] = {1,2,3,4,5}; /* num[1][2] = 0*/
```

```
int num[2][3] = {{1,2,3},{1,2,3}}; /* row elements are initialized separately*/
```

```
int num[2][3] = {{1,2},{4}}; /* num[0][2] = 0 num[1][1]=num[1][2]=0*/
```

Example for 4-dimensional array:

```
sales [year] [month] [area] [salesperson]
```

Advantages of arrays:

- Simple and easy to use.
- Stored in Contiguous locations.
- Fast retrieval because of its indexed nature.
- No need to worry about the allocation and de-allocation of arrays.

Limitations

- Conventional arrays are static in nature. Memory is allocated in the beginning of the execution. If m elements are needed, out of n locations defined, n-m locations are unnecessarily wasted.
- No automatic array bounds checking during compilation.

Now, let us take up a simple problem of declaring, initializing and displaying the elements of an array of data-type integer.

```
#include<stdio.h>
int main()
{
    int i;
    int arr[5] = {10,20,30,40,50};

    // declaring and Initializing array in C
    //To initialize all array elements to 0, use int arr[5]={0};
    /* Above array can be initialized as below also
    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;
    arr[3] = 40;
    arr[4] = 50; */

    for (i=0;i<5;i++)
    {
        // Accessing each variable
        printf("value of arr[%d] is %d \n", i, arr[i]);
    }
    return 0;
```

}

Let us take a problem of finding first 20 Fibonacci numbers using arrays, given that first two numbers as 0 and 1. Initially, let us analyse the problem. We all know that Fibonacci numbers can be generated by adding previous two numbers in the sequence of numbers assuming that first two numbers given as 0 and 1. Hence we have the following number sequence as Fibonacci numbers.

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, and so on.

The program can be written as follows.

```
/* Program to calculate the first 20 Fibonacci numbers. */
```

```
#include <stdlib.h>
#include <stdio.h>
int main()
{
    int i, fibonacci[ 20 ];

    fibonacci[ 0 ] = 0;
    fibonacci[ 1 ] = 1;

    for( i = 2; i < 20; i++ )
        fibonacci[ i ] = fibonacci[ i-2 ] + fibonacci[ i-1 ];

    for( i = 0; i < 20; i++ )
        printf( "Fibonacci[ %d ] = %f\n", i, fibonacci[ i ] );
    return 0;
} /* End of Fibonacci program */
```

Chapter 8: User-defined Functions

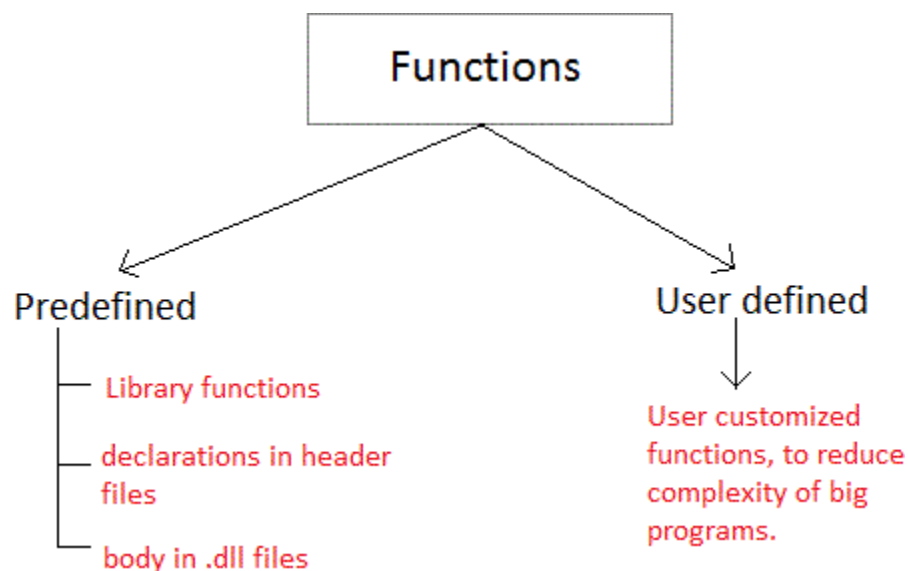
A function is a self-contained block of code that performs a particular task.

There are many situations where we might need to write same line of code for more than once in a program. This may lead to unnecessary repetition of code, bugs and even becomes boring for the programmer. So, C language provides an approach in which you can declare and define a group of statements once in the form of a function and it can be called and used whenever required.

These functions defined by the user are also known as User-defined Functions

C functions can be classified into two categories,

1. Library functions
2. User-defined functions



Library functions are those functions which are already defined in C library, example `printf()`, `scanf()`, `strcat()` etc. You just need to include appropriate header files to use these functions. These are already declared and defined in C libraries.

A User-defined functions on the other hand, are those functions which are defined by the user at the time of writing program. These functions are made for code reusability and for saving time and space.

BENEFITS OR NEED OF USING FUNCTIONS

1. It provides modularity to your program's structure.
2. It makes your code reusable. You just have to call the function by its name to use it, wherever required.
3. In case of large programs with thousands of code lines, debugging and editing becomes easier if you use functions.
4. It makes the program more readable and easy to understand.
5. There are times when some types of operation or calculation is repeated at many points throughout a program. So functions saves both time and space.

A function may be used by many other programs, this means that a C program can build on what others have already done, instead of starting over, from scratch.

ELEMENTS OF USER-DEFINED FUNCTIONS

There are three elements related to functions

- Function definition
- Function call
- Function declaration

Function Declaration:

General syntax for function declaration is,

returntype functionName(type1 parameter1, type2 parameter2,...);

Like any variable or an array, a function must also be declared before its used. Function declaration informs the compiler about the function name, parameters is accept, and its return type. The actual body of the function can be defined separately. It's also called as **Function Prototyping**.

Function declaration consists of 4 parts.

- `returntype`
- `function name`
- `parameter list`
- `terminating semicolon`

returntype

When a function is declared to perform some sort of calculation or any operation and is expected to provide with some result at the end, in such cases, a return statement is added at the end of function body. Return type specifies the type of value(int, float, char, double) that function is expected to return to the program which called the function.

Note: In case your function doesn't return any value, the return type would be void.

functionName

Function name is an identifier and it specifies the name of the function. The function name is any valid C identifier and therefore must follow the same naming rules like other variables in C language.

parameter list

The parameter list declares the type and number of arguments that the function expects when it is called. Also, the parameters in the parameter list receives the argument values when the function is called. They are often referred as **formal parameters**.

Function definition:

The function definition is an independent program module that is specially written to implement the requirements of the function.

The general syntax of function definition is,

```
returntype functionName(type1 parameter1, type2 parameter2,...)
{
    // function body goes here
}
```

The first line `returntype functionName(type1 parameter1, type2 parameter2,...)` is known as **function header** and the statement(s) within curly braces is called **function body**.

Note: While defining a function, there is no semicolon(;) after the parenthesis in the function header, unlike while declaring the function or calling the function.

functionbody

The function body contains the declarations and the statements(algorithm) necessary for performing the required task. The body is enclosed within curly braces { ... } and consists of three parts.

- **local** variable declaration(if required).
- **function statements** to perform the task inside the function.
- a **return** statement to return the result evaluated by the function(if return type is void, then no return statement is required).

Calling a function:

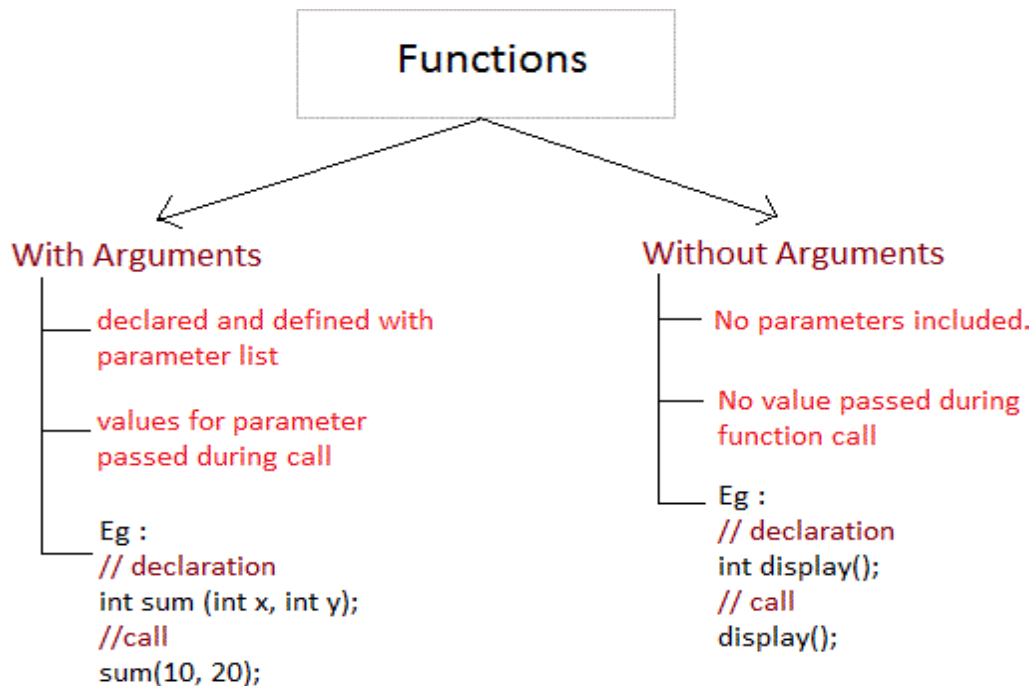
When a function is called, control of the program gets transferred to the function.

functionName(argument1, argument2,...);

In the example program, the statement multiply(i, j); inside the main() function is function call.

PASSING ARGUMENTS TO A FUNCTION

Arguments are the values specified during the function call, for which the formal parameters are declared while defining the function.



It is possible to have a function with parameters but no return type. It is not necessary, that if a function accepts parameter(s), it must return a result too.

```
#include<stdio.h>

int multiply(int a, int b);

int main()
{
    ... ..
    result = multiply(i, j);
    ... ..
}

int multiply(int a, int b)
{
    ... ..
}
```



providing arguments while calling function

While declaring the function, we have declared two parameters a and b of type int. Therefore, while calling that function, we need to pass two arguments, else we will get compilation error. And the two arguments passed should be received in the function definition, which means that the function header in the function definition should have the two parameters to hold the argument values. These received arguments are also known as **formal parameters**. The name of the variables while declaring, calling and defining a function can be different.

Returning a value from function:

A function may or may not return a result. But if it does, we must use the return statement to output the result. return statement also ends the function execution, hence it must be the last statement of any function. If you write any statement after the return statement, it won't be executed.

```

#include<stdio.h>

int multiply(int a, int b);

int main()
{
    ... ..
    result = multiply(i, j);
    ... ..
}

int multiply(int a, int b)
{
    ... ..
    return a*b;
}

```



The value returned by the function must be stored in a variable.

The datatype of the value returned using the return statement should be same as the return type mentioned at function declaration and definition. If any of it mismatches, you will get compilation error.

RETURN VALUES AND THEIR TYPES

A function may or may not send back any value to the calling function or The called function can only return one value per call.

SYNTAX:

return;

or

return (expression);

return;

–Plain return does not return any value

- Acts as the closing brace of the function
- The control is immediately passed back to the calling function

EXAMPLE:

```
if(error)
return;
```

return (expression);

- Return the value of the expression

EXAMPLE:

```
mul(int x,int y)
{
int p;
p = x*y;
return(p);
}
```

A function may have more than one return statements.

EXAMPLE:

```
fun()
{
if(x <= 0)
return(0);
else
return(1);
}
```

There will be times when we may find it necessary to receive the character, float or double type of data. We must explicitly specify the return type on both the function definition and the prototype declaration. The type-specifier tells the compiler, the type of data the function is to return. The called function must be declared at the start of the body in the calling function, like any other variable. This is to tell the calling function the type of data that the function is actually returning.

TYPE OF USER-DEFINED FUNCTIONS IN C

There can be 4 different types of user-defined functions, they are:

1. Function with no arguments and no return value
2. Function with no arguments and a return value
3. Function with arguments and no return value
4. Function with arguments and a return value

Function with no arguments and no return value:

Such functions can either be used to display information or they are completely dependent on user inputs.

Below is an example of a function, which takes 2 numbers as input from user, and display which is the greater number.

```
#include<stdio.h>
void greatNum();    // function declaration

int main()
{
    greatNum();    // function call
    return 0;
}

void greatNum()    // function definition
{
    int i, j;
    printf("Enter 2 numbers that you want to compare...");
    scanf("%d%d", &i, &j);
    if(i > j) {
        printf("The greater number is: %d", i);
    }
    else {
        printf("The greater number is: %d", j);
    }
}
```

Function with no arguments and a return value:

We have modified the above example to make the function `greatNum()` return the number which is greater amongst the 2 input numbers.

```
#include<stdio.h>
int greatNum();    // function declaration

int main()
{
    int result;
    result = greatNum();    // function call
    printf("The greater number is: %d", result);
    return 0;
}

int greatNum()    // function definition
{
    int i, j, greaterNum;
    printf("Enter 2 numbers that you want to compare...");
    scanf("%d%d", &i, &j);
    if(i > j) {
        greaterNum = i;
    }
    else {
        greaterNum = j;
    }
    // returning the result
    return greaterNum;
}
```

Function with arguments and no return value:

We are using the same function as example again and again, to demonstrate that to solve a problem there can be many different ways.

This time, we have modified the above example to make the function `greatNum()` take two int values as arguments, but it will not be returning anything.

```

#include<stdio.h>
void greatNum(int a, int b);    // function declaration

int main()
{
    int i, j;
    printf("Enter 2 numbers that you want to compare...");
    scanf("%d%d", &i, &j);
    greatNum(i, j);    // function call
    return 0;
}

void greatNum(int x, int y)    // function definition
{
    if(x > y) {
        printf("The greater number is: %d", x);
    }
    else {
        printf("The greater number is: %d", y);
    }
}

```

Function with arguments and a return value:

This is the best type, as this makes the function completely independent of inputs and outputs, and only the logic is defined inside the function body.

```

#include<stdio.h>
int greatNum(int a, int b);    // function declaration

int main()
{
    int i, j, result;
    printf("Enter 2 numbers that you want to compare...");
    scanf("%d%d", &i, &j);
    result = greatNum(i, j); // function call
    printf("The greater number is: %d", result);
}

```

```

    return 0;
}

int greatNum(int x, int y)    // function definition
{
    if(x > y) {
        return x;
    }
    else {
        return y;
    }
}

```

Nesting of Functions:

C language also allows nesting of functions i.e to use/call one function inside another function's body. We must be careful while using nested functions, because it may lead to infinite nesting.

```

function1()
{
    // function1 body here

    function2();

    // function1 body here
}

```

If function2() also has a call for function1() inside it, then in that case, it will lead to an infinite nesting. They will keep calling each other and the program will never terminate.

Not able to understand? Lets consider that inside the main() function, function1() is called and its execution starts, then inside function1(), we have a call for function2(), so the control of program will go to the function2(). But as function2() also has a call to function1() in its body, it will call function1(), which will again call function2(), and this will go on for infinite times, until you forcefully exit from program execution.

TYPES OF FUNCTION CALLS IN C

Functions are called by their names, we all know that, then what is this tutorial for? Well if the function does not have any arguments, then to call a function you can directly use its name. But for functions with arguments, we can call a function in two different ways, based on how we specify the arguments, and these two ways are:

1. Call by Value
2. Call by Reference

Call by Value:

Calling a function by value means, we pass the values of the arguments which are stored or copied into the formal parameters of the function. Hence, the original values are unchanged only the parameters inside the function changes.

```
#include<stdio.h>
void calc(int x);

int main()
{
    int x = 10;
    calc(x);
    // this will print the value of 'x'
    printf("\nvalue of x in main is %d", x);
    return 0;
}

void calc(int x)
{
    // changing the value of 'x'
    x = x + 10 ;
    printf("value of x in calc function is %d ", x);
}
```

value of x in calc function is 20 value of x in main is 10

In this case, the actual variable x is not changed. This is because we are passing the argument by value, hence a copy of x is passed to the function, which is updated during function execution, and that copied value in the function is destroyed when the function ends(goes out of scope). So the variable x inside the main() function is never changed and hence, still holds a value of 10.

But we can change this program to let the function modify the original x variable, by making the function calc() return a value, and storing that value in x.

```
#include<stdio.h>
int calc(int x);

int main()
{
    int x = 10;
    x = calc(x);
    printf("value of x is %d", x);
    return 0;
}

int calc(int x)
{
    x = x + 10 ;
    return x;
}
value of x is 20
```

Call by Reference:

In call by reference we pass the address(reference) of a variable as argument to any function.

When we pass the address of any variable as argument, then the function will have access to our variable, as it now knows where it is stored and hence can easily update its value.

In this case the formal parameter can be taken as a **reference** or a **pointer**(don't worry about pointers, we will soon learn about them), in both the cases they will change the values of the original variable.

```

#include<stdio.h>
void calc(int *p);    // functin taking pointer as argument

int main()
{
    int x = 10;
    calc(&x);    // passing address of 'x' as argument
    printf("value of x is %d", x);
    return(0);
}

void calc(int *p)    //receiving the address in a reference pointer variable
{
    /*
        changing the value directly that is
        stored at the address passed
    */
    *p = *p + 10;
}

```

value of x is 20